

Deliverable 2

SENG3011 22T1

Design Details

Team f0b5 – Matthew Brian, Jaeff Hong, Mathieu Paturel, Nicholas Ramsay and Huiyi Wang

(z5330484, z5316653, z5312157, z5367013, z5261844)

Contents

API Design Details	3
Final Architecture.....	3
API Technologies.....	3
API Structure	4
Libraries and Languages (Choice of Implementation).....	4
Primary Language.....	4
Database Type	4
Database ORM.....	4
Server Endpoints.....	5
Server Proxy.....	5
Scraper.....	5
Parser (Natural Language Processing)	5
Linode.....	6
Deployment Pipeline.....	6
Swagger	6
Challenges Addressed and Shortcomings.....	6
Deployment.....	7
Production Capabilities.....	7
Natural Language Processing	8
Documentation.....	8

API Design Details

Describe final architecture of your API, justify the choice of implementation, challenges addressed and shortcomings, in your Design Details report in your Confluence document. You may update management information if necessary.

Most of our implementation decisions have remained consistent from what we submitted in Deliverable 1. When you see a decision that differs from, or extends, what has previously been communicated, the new technology we are using will be **highlighted as follows**.

Firstly, we will discuss our final architecture. Then we will look at the pros and cons of the languages and libraries we will use to implement that architecture. Finally, we will reference the challenges and shortcomings we have experienced in setting up our API, and what we have done to overcome them.

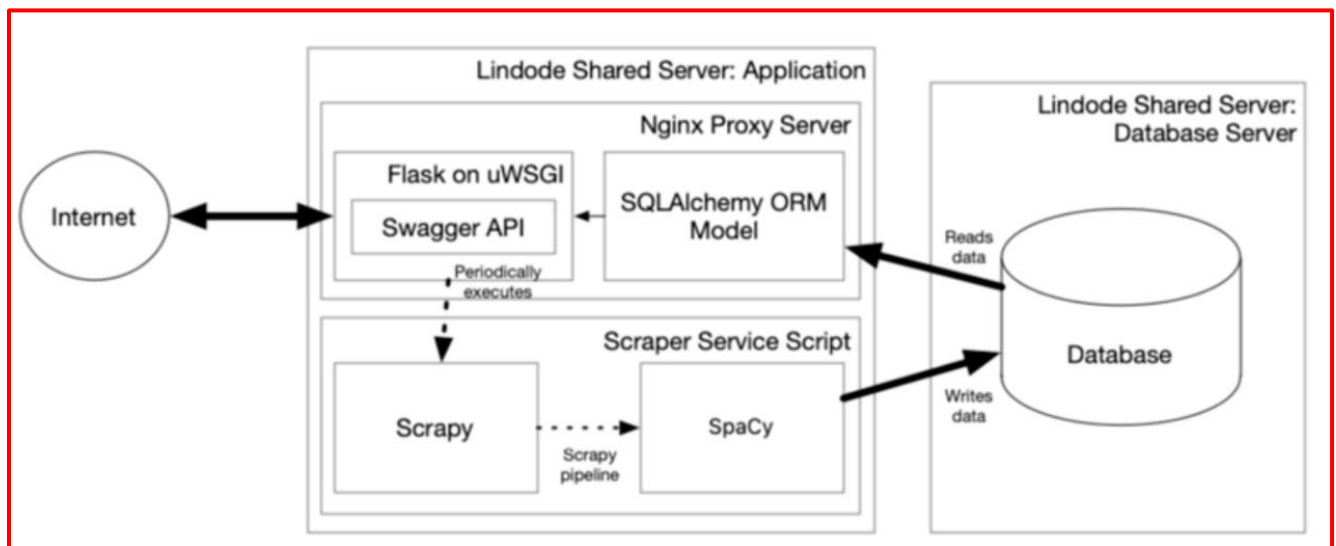
Final Architecture

API Technologies

Our final architecture is mostly as was described in Deliverable 1. We are using **Python** to program our business logic, and **MySQL** for our database. The API is operated using the REST calling convention.

The API runs on a **Flask** server which is opened to the web via the **uWSGI** web application layer. This in turn is exposed to the internet in a performant way by the **NGINX** proxy server. The endpoints available over the API are viewable in **Swagger** documentation. A **Scrapy** microservice periodically hands information from the CIDRAP website off to our **SpaCy** parser, aided by the **GeoNames** 'geoparsing' library, which then populates the database.

This is all deployed to a **Linode** shared server hosted in Sydney. This is done automatically using a **GitHub Actions** pipeline with a custom **ssh-script**.



API Structure

The API is described in much greater detail than I could justify here, so *I am once again asking you* to check the Swagger documentation for our up-to-date API structuring. ([https://app.swaggerhub.com/apis/tanyawhy/SENG3011_f0b5/1.0.0#/.](https://app.swaggerhub.com/apis/tanyawhy/SENG3011_f0b5/1.0.0#/))

The API doesn't need to be complex, and ours isn't, offering *three* resource-oriented endpoints to fetch requested data. This structuring allows us to add more queries in the future if required.

Most of our choices as justified below were made because of prior experience, cost-effectiveness, language integration, and course recommendations. Experience was the most important factor, as we needed to get started quickly on development and don't have time to waste on learning the syntax of a new language. In a lot of cases there may have been technical benefits in using another package, language, or paradigm to solve a problem, but they were never worth the time we would be required to invest to learn.

Libraries and Languages (Choice of Implementation)

Primary Language

There has been no change from D1 as to our reasoning on choosing **Python**. The whole team is experienced with it, there are many third-party libraries that integrate with it, and its widespread adoption will make it easier to maintain and update in the future. Other languages were avoided primarily due to our lack of experience with them, despite potential speed or other benefits that could be derived from using them in production.

Database Type

There has been no change from D1 in choosing **MySQL** to run our database. The information we are required to store is apt to store in a relational database instead of a document-oriented one as we know in advance what we need to store. Again, we are also all highly experienced with SQL, which has made it faster to start development. MySQL is also widely supported and is the most beginner friendly to use, with a focus on simplicity, speed, and reliability. Other database packages like PostgreSQL share increasingly similar features to MySQL nowadays, so we selected MySQL for its wide community support.

Database ORM

There has been no change from D1 in our choice of **SQLAlchemy** to manage our flask-to-database interactions. This prevents bad actors from entering malicious requests into our database and communication with it from Python more efficient. The package also has a tight integration with Flask, easing development. Researching which package to use for this task yielded SQLAlchemy as the clear leader in Python-to-SQL interactions, combining both simplicity, speed, and community support.

Libraries and Languages continues on the next page

Server Endpoints

There has still been no change from D1 as to our choice of server. **Flask** allows us to create, in Python, a basic server architecture. We all have extensive experience with it, having used it to develop in SENG2021 and COMP1531.

Server Proxy

The choice of **uWSGI** and **NGINX** remains the same from D1. They provide us with a *much more* performant intermediary between REST requests and the Flask API. These were selected due to their tight integration with the Flask platform, and focus on simplicity, invaluable in a small student project.

Scraper

We had to choose a package that would allow us to retrieve data effectively from the CIDRAP website. The choice of **Scrapy** has not changed from D1. It is a Python package and thus integrates tightly with the other packages we have chosen. We selected Scrapy specifically due to its focus on web crawling capabilities. This is opposed to *BeautifulSoup* which focuses on extracting specific elements from pages and *Selenium* which focuses on page interaction. This was integrated with the Python package **APScheduler** which allows us to run the scraping script once a day.

Parser (Natural Language Processing)

After scraping articles from CIDRAP we had to be able to parse the data we need out of it. This is a complex linguistic exercise which is highly dependent on sentence structuring and relationships between words. Thus, we needed to select a robust Python package that could assist in solving this complex problem.

After careful consideration we decided to use **SpaCy**. This is for several reasons. Firstly, it is designed to be fast, which is important given the number of articles we plan on parsing. Secondly, it is easy to use, and has many modules that are aimed at more specific use cases like ours. Finally, it has used neural networks to train its NLP models making them more accurate. A drawback of this package is the limited number of languages it supports, though we only plan on parsing English, so this is not a problem. Our use case isn't highly specialised which makes SpaCy a better choice than something like Natural Language Toolkit (NLTK) for Python, which would take more time to set up and train to solve our specific problem.

Furthermore, we needed to extract locations from scraped articles and have them be converted to a hierarchical representation. We decided on using the **GeoNames** representation standard as it integrates well with SpaCy.

Libraries and Languages continues on the next page

Linode

Our server is still deployed on a **Linux** server in Sydney, Australia and we do not plan to change this. **Linode's** transparent pricing model is easy for us to understand and interact with, giving peace of mind as to costs (invaluable to university students). This is unlike other cloud providers like AWS and Azure which would require much more mental capital to be expended to understand and setup programs on.

We have decided to use Linux as our operating system as we have had the most experience with it developing at university.

Deployment Pipeline

We needed to create a CD pipeline to deploy our project more easily to the Linode server. None of us have ever had experience with this area of programming and having used CI before in **GitHub Actions** we decided to use this for deployment. This is relatively simple and could be expanded upon if required. On a successful PR into the main branch, GitHub uses the custom action **ssh-script** to access the Linode server and pull changes down. It then restarts the Flask/uWSGI/NGINX server processes using **systemctl** and reruns the scraper/parser. Once again, this choice was made as it was part of a system we had experience with before. Other systems like Jenkins or Bamboo may have been better choices technically, but this is succinct and simple way for it to be done using systems we have used before.

Swagger

The choice of **Swagger** has not changed since D1. We are finding it good to use, and it has integrated well with our development pipeline.

Challenges Addressed and Shortcomings

One of the biggest challenges we had to address coming into this project was deployment in a real production environment. We know that there is the potential for other teams to use our API in the next deliverable. This meant extending outside our collective comfort zones to do several things.

Firstly, we must interact with and setup operations on a real cloud server rather than a local testing environment. This includes setting up an automated deployment system for changes we make.

Secondly, we must add production level capabilities to the operation of the API, for the first time for most of us considering both performance and security.

Thirdly, we were required to implement language processing to extract information intelligently from CIDRAP.

Finally, we had to set up public facing API documentation that was extensive enough that other teams would be able to interact with our database with confidence.

Deployment

The first big issue here was selecting a cloud provider. While we had heard of platforms like AWS and Azure, and have attended SENG lectures on their usage, they still seem incredibly intimidating. The wide array of microservices they offer and their incredibly opaque pricing model made platform choices difficult to reason about with confidence. We overcame this when we discovered Linode, which provided something understandable and confidently cost-effective, important for development with minimal budget and time. This was all inside a Linux server with a guaranteed amount of uptime, specifications, network bandwidth. This checked our boxes of cost-effectiveness and transparency, so we decided to use it. There have been no issues with it so far.

Second we had to set up a development pipeline where changes to our codebase could be made locally and then automatically deployed to production. This was an entirely new world to all of us. Though we had some experiences with GitHub pipelines before, we'd never used one to deploy directly to the cloud. Fortunately, it was easily extended to do automatically deploy in what we assume is a hacky way as you can read on previous pages. If it works, it works.

Production Capabilities

Our team members have only worked on programs that have operated in a local environment. None of us have deployed a program to the cloud before with the potential for it to be used by so many other people, intended or not. As has been previously mentioned, this means we needed to consider the security of our platform.

We overcame this by enabling the default Linux firewall and only allowing communication with the Linode server over ports we allowed. We also added in an SQLAlchemy intermediary between REST requests and database interaction to mitigate SQL injection attacks.

Unfortunately, none of us have taken security courses and are unaware of subtler issues which may be present in our server configuration that make it vulnerable to bad actors. If we were to make this API public and viewable to those outside SENG3011 we would need to look at this with much great scrutiny.

Furthermore, it also meant we needed to consider performance. Our application needs to be able to handle many queries simultaneously, and a standard Flask server being exposed directly to the internet was unsuitable for this. As such we set up Flask to use the uWSGI interface. We then exposed this to the internet via a NGINX proxy server which allows us to make performant and asynchronous server requests. These two technologies combined allowed us to overcome the challenge of making performant requests to Flask.

Challenges and Shortcomings continues on the next page

Natural Language Processing

This is the heart of our API, and the hardest part to implement. So, it is unfortunate that none of our team members had prior experience with linguistics or machine learning to make this easier. We had to be able to parse infinitely variable sentence structures and determine the relationships between diseases and syndromes.

The problem seemed very daunting at first. Fortunately for us, it appears *many* people have had this issue before and greatly simplified it for us. The SpaCy library presented all required functionality and more in a very beginner-friendly interface.

Documentation

No team members had experience with an automated documentation platform like Swagger before this course. Nonetheless we assigned Tanya to research how convenient such a platform would be to use. She found it to be very easy, allowing us to overcome the challenge of making documentation *very quickly*.